

# Welcome to Durion — Project Onboarding

---

*A first-draft guide for human experience engineers and project members. This document will grow as we learn together.*

---

## What Are We Building?

Durion is an open-source platform for managing tire service and auto repair operations. It covers the full shop workflow — from the moment a customer walks through the door to the moment they drive away with a completed invoice in hand.

Think of it as the operating system for a tire shop or auto service location: checking in customers, building estimates, dispatching technicians, tracking parts inventory, collecting payment, and keeping the books.

The software is designed to serve operations of all sizes — from a single-location tire shop to a multi-site fleet service operation. A key design goal is that every location, large or small, should get enterprise-grade software that was previously only affordable to the biggest chains.

---

## Why Are We Doing This?

### The Problem

The tire and auto service industry is fragmented. Shops and fleets run on a patchwork of aging, vendor-locked software. Switching vendors is painful, data is siloed, and most providers charge fees that eat into already-thin margins. Small and mid-size operators are especially disadvantaged — they can't afford the custom integrations that large chains write into their contracts.

### The Mission

**Durion** is the flagship software project of the **Tire Industry Open Technology Foundation (TIOTF)**, a Delaware 501(c)(6) non-profit trade association. TIOTF's mission is to advance open, vendor-neutral technology standards and software for the tire, fleet, and service-provider industries.

By building open source, we change the economics. Anyone can adopt Durion. No one owns the lock-in. The industry benefits collectively.

TIOTF brings together tire manufacturers, service providers, fleets, technology companies, and independent contributors as members. Governance flows through the membership — decisions about the software and its standards are made collectively, not by a single vendor.

**Durion Solutions, Inc.** — a wholly-owned subsidiary of TIOTF — provides the commercial layer: support contracts, hosting, training, and integration services for organizations that need them. This model keeps the software genuinely open while giving the Foundation sustainable operating revenue.

---

## How We Are Building It — The Experimental Approach

We are building Durion incrementally, and we are treating every release as an opportunity to learn.

## Capabilities, Not Big Releases

Rather than designing the entire system upfront and shipping it all at once, we break work into discrete **capabilities** — self-contained pieces of functional behavior that a real user could actually use. Each capability is small enough that we can get real feedback on it without waiting months.

We group related capabilities into focused execution sprints that build out one domain area at a time. Each sprint teaches us something:

- Does this workflow match how shops actually operate?
- Are we missing an edge case that matters to real users?
- Is there a constraint we discovered through building that we need to reconsider?

**The prototype is not a demo. It is an instrument for discovering what we do not yet know.**

We use it to reveal functional design gaps before real customers depend on the system. Requirements written on paper are always incomplete. Prototypes expose the gaps. We fix them before they become production problems.

## One Customer, One Isolated Cell

Our deployment model is designed so that each paying customer gets their own isolated runtime environment. This matters for two reasons:

1. It protects customer data strictly — one shop's data is never mixed with another's.
2. It gives us clean signal. When something breaks or a workflow doesn't fit, we know exactly which customer context produced it.

This model also means we can give early customers a production-grade experience even while we are still learning. We are not asking them to share unstable infrastructure.

## What "Incremental Experiments" Means in Practice

Our approach draws directly from the disciplines of **Lean Startup** and **Agile** product development. These are not buzzwords — they represent a fundamentally different way of thinking about risk in software projects.

### **The Core Problem: We Don't Know What We Don't Know**

Traditional software projects begin with a long requirements phase, build everything, then ship. The failure mode is predictable: by the time real users touch the software, the assumptions baked into the design have often drifted far from reality. You have spent months and money building the wrong thing.

The alternative — which we use — is to treat every release as a **structured experiment whose purpose is to reduce uncertainty**.

## Validated Learning

Eric Ries, in *The Lean Startup*, introduced the concept of **validated learning**: the idea that the most valuable output of an early-stage product is not software — it is *knowledge*. Specifically, knowledge that is confirmed

(or refuted) by real user behavior, not by internal opinion.

Every capability we ship is designed to validate or invalidate a hypothesis about how the market works. We write the hypothesis down before we build. After we ship, we ask: did the market behave the way we predicted? If not, what did we learn?

Examples of hypotheses we are currently testing:

- *We believe shops want to capture digital customer approval before starting work.* If early customers print and sign paper instead — even when the digital option is right in front of them — we have learned something important about adoption friction.
- *We believe a Dispatcher manages all appointment conflicts from a desktop.* If we observe Dispatchers walking the floor with a phone trying to communicate conflicts verbally, we have evidence that a mobile interface matters more than we assumed.
- *We believe per-location pricing governance is the right model for multi-site operators.* If early multi-site customers ask immediately for a central override, we learn that our model does not reflect how they actually think about pricing authority.

## Minimum Viable Products (MVPs)

In Lean and Agile, an **MVP — Minimum Viable Product** — is the smallest version of a capability that still delivers real value and enables real learning. It is not a rough prototype. It is a deliberately scoped, production-quality piece of work that is intentionally limited so that we can learn fast without over-engineering.

We use MVPs across every domain:

- The first version of the estimate builder does not include every pricing rule. It includes enough to let a Service Advisor build a valid quote. We observe what is missing when they try to use it with a real customer.
- The first version of the work order execution flow does not include every technician communication pattern. It includes enough to let a Technician pick up work, update progress, and complete the job. We discover what friction points real technicians encounter before we invest further.

An MVP shifts the risk profile of the project. Instead of asking "*did we build everything?*", we ask "*did we learn enough to justify the next investment?*"

## The Build–Measure–Learn Loop

The Lean Startup describes a tight feedback cycle: **Build** something small → **Measure** how real users respond → **Learn** what to do next. Then repeat.

For Durion, this loop looks like:

1. **Identify the riskiest assumption** in the next domain area — the thing we are most likely to be wrong about.
2. **Build the MVP** that makes that assumption testable with a real user.
3. **Observe** — do customers and shop staff use the capability the way we expected? Do they avoid parts of it? Do they ask for something we didn't build?
4. **Update the design** based on evidence, not opinion, before investing in the next sprint.

This is not a slow process. The discipline of keeping MVPs genuinely minimal means we run these loops quickly. The goal is not perfection on the first pass — it is moving from assumption to evidence as fast as possible.

What This Means for Human Experience Engineers

The most valuable thing human experience engineers can contribute to this process is **domain knowledge and honest feedback**. If a workflow we have built does not match how shops actually operate, the earlier we hear that, the better. A comment in a review meeting is infinitely cheaper than discovering the gap after deployment.

When you review a prototype or a demonstration, the most useful question you can bring is: *"In my experience, would a real shop actually do it this way?"* That question is the input the build–measure–learn loop depends on.

---

Who Uses Durion? — The Personas

Durion is designed around the real people who work in tire and auto service environments. We call these people **personas** — they represent the distinct roles and responsibilities inside a shop.

Understanding personas matters because each person has a different job to do, different information they need, and a different screen they will interact with. The software is organized around their workflows, not around database tables.

Customer-Facing Roles

| Persona                    | What They Do                                                                                                                                                                                                             |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Service Advisor            | Greets customers, captures vehicle and service needs, builds estimates, routes work to technicians, and manages the work order from intake through completion. The dominant front-counter role for complex service work. |
| Customer Support Associate | Handles lighter-touch intake, front-desk greeting, customer lookup, and end-of-transaction checkout including invoice review and payment collection. Often the first face a customer sees.                               |
| Account Manager            | Manages ongoing commercial and fleet customer relationships — long-term accounts, special pricing arrangements, and relationship-level CRM activity.                                                                     |

Shop Operations

| Persona          | What They Do                                                                                                                                                                     |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Location Manager | Runs the shop — directs service advisors, approves estimates above threshold, handles escalations, manages technician assignments, and owns shop-level financial accountability. |
| Dispatcher       | Schedules appointments, manages the daily shop schedule, coordinates technician assignments, and handles mobile dispatch for operations with field technicians.                  |

## Shop Execution

| Persona | What They Do | | -- -| --- | | **Technician** | Performs the physical repair or service work. Views their assigned work orders, records progress, requests additional work approval, issues or returns parts, and signals completion. | | **Parts Manager / Parts Associate** | Manages parts availability and sourcing, fulfills picks against work orders, handles returns-to-stock, and supports the technician execution loop from the parts counter. |

## Inventory and Warehouse

| Persona                          | What They Do                                                                                                                                              |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Inventory Control Manager</b> | Owns inventory accuracy — plans cycle counts, reviews adjustments, governs stock control policies, and monitors inventory health.                         |
| <b>Warehouse Associate</b>       | Performs physical stock movements — receiving goods, running put-away tasks, fulfilling replenishment, and executing cycle counts on the warehouse floor. |

## Finance and Accounting

| Persona                                     | What They Do                                                                                                               |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Accounting Associate</b>                 | Handles day-to-day accounting operations — invoice review, AR application, payment recording, and accounting event triage. |
| <b>Accounting Manager / Finance Manager</b> | Manages period close, financial review, and accounting governance. Oversight of the accounting team.                       |

## The Customer Journey — How It All Connects

A single vehicle visit touches nearly every persona in the list above. That end-to-end flow is what we call a **journey**. We have mapped out the major journeys that the software must support.

### Journey 1: Customer Intake to Estimate

A customer arrives with a service need. The **Service Advisor** pulls up their customer record, retrieves vehicle history, and opens a work order. They build an estimate with labor and parts, applying pricing rules automatically. The estimate goes to the customer for review and approval — digitally captured with signature if needed. Once approved, the estimate is promoted into an executable work order.

**Key moments:** customer lookup → draft estimate → pricing → customer approval → work order creation

### Journey 2: Scheduling, Dispatch, and Shop Coordination

Before or after intake, the estimate or appointment needs to be slotted into the shop's schedule. The **Dispatcher** or **Location Manager** assigns technicians, bays, and time slots. Mobile technicians can be reached through a dispatch-capable interface. Conflicts surface and can be resolved before work begins.

**Key moments:** appointment creation → schedule management → technician assignment → visibility for all parties

### Journey 3: Work Order Execution and Completion

The **Technician** receives their assigned work and starts executing. They record progress, flag additional findings, request parts from the Parts counter, and handle part substitutions when stock doesn't match. When work is complete, the Service Advisor reviews, resolves any approval-gated changes, and hands the completed work order to billing.

**Key moments:** technician picks up work → executes and updates → parts issued/returned → work completed → billing handoff

### Journey 4: Inventory, Warehouse, and Fulfillment

Parts don't appear by magic. Behind every work order is a supply chain: purchase orders sent to distributors, goods received into staging, parts put away into bins, stock counts kept accurate through cycle counts. The **Warehouse Associate** and **Inventory Control Manager** own the accuracy of what is on the shelf. When a Technician needs a part, the pick and fulfillment process connects the warehouse floor to the shop floor.

**Key moments:** PO → receiving → put-away → replenishment → pick request → consumed or returned

### Journey 5: Billing, Accounting, and Finance

When work is complete, the **Customer Support Associate** processes the final invoice — reviewing line items, collecting payment, handling exceptions like voids or refunds. The accounting team receives the resulting financial events, applies payments to AR, and routes journal entries. Finance managers handle period close and reporting.

**Key moments:** invoice ready → customer payment → AR application → journal entry → period close

### Journey 6: Product, Pricing, and Promotions Governance

Before any estimate can be built, someone needs to have defined the products in the catalog, set the pricing rules, and configured any promotions. **Pricing Administrators** and **Product Administrators** maintain this layer. Their work enables Service Advisors to quote correctly without having to remember which rule applies.

**Key moments:** product setup → price book → pricing rules → promotions → downstream quoting

### Journey 7: Customer Relationship Management

Underlying all of the above is a customer record. Parties (people and businesses), vehicles, and their relationships are managed in the CRM layer. Account Managers and Service Advisors maintain this record over time, making every future visit faster and more accurate.

**Key moments:** person/account creation → vehicle association → relationship stewardship → historical context available at intake

---

## What We Have Built So Far

The platform has delivered capabilities across all of the major operational domains a shop needs:

- **Work execution** — estimates, work orders, approval, completion, and invoicing
- **Accounting** — event ingestion, posting rules, accounts receivable, accounts payable, and credit memos
- **Shop management** — locations, appointments, dispatch, timekeeping, and scheduling
- **People and identity** — RBAC, CRM, work order integration, location assignment, and location topology
- **Product and pricing** — product master data, cost management, and pricing rules
- **Inventory** — stock ledger, receiving, put-away, cycle counts, allocations, and purchasing

This is a substantial foundation. Every one of these capabilities represents a real workflow that a real person in a shop can run through.

---

## The SDK — Connecting Front End to Back End

Durion is split into two distinct layers: the **backend** (where business logic, data, and rules live) and the **frontend** (the screens and interfaces that users interact with). These two layers communicate through a purpose-built **SDK** — a software development kit that defines exactly what information the frontend can request and what the backend will return.

The SDK is the contract between them. When we change something in the backend, the SDK update makes the change visible and usable by the frontend in a predictable, controlled way. When a third-party developer or partner wants to build on top of Durion, the SDK is the starting point.

For human experience engineers: think of the SDK as the language the two halves of the software use to talk to each other. It keeps the system coherent and makes it harder for a change in one place to accidentally break something in another.

---

## What We Are Still Learning

We are candid that there are open questions this project is designed to answer:

**About workflows:** Do real technicians need a mobile pick-execution interface, or do they hand off parts requests to a Parts Associate? We have documented the uncertainty and are deferring the decision until we have shop-floor observation data.

**About integrations:** Which external data providers matter most in the first markets we enter? Tire distributor availability? OEM vehicle specifications? Fleet telematics? We have designed the integration boundary to be neutral — we are not betting on any single vendor yet.

**About pricing:** Does per-location pricing governance match how multi-site operators actually want to manage pricing, or do they prefer a centralized model? We have built both and will learn from early adopters which they prefer.

**About the market:** Which types of operations will value open-source most — independent owner-operators, franchise networks, fleet operators, or specialty programs? We do not want to over-index on one segment before we have real signal.

Every capability delivery sprint is partly a market experiment. The prototype is the instrument. Real user behavior is the data.

## How You Can Contribute (Without Writing Code)

Human experience engineers are essential to this project succeeding. The most valuable contributions at this stage are:

- **Domain expertise:** If you have worked in or with tire service or fleet operations, your knowledge of real workflows is more valuable than any design document. Where we have a question, we will ask you.
- **Customer introductions:** The prototype needs real users. If you have relationships with shop operators, fleet managers, or service chain decision-makers, connecting us to those conversations accelerates everything.
- **Feedback on journeys:** Walk through the journey descriptions above. Tell us what is missing, what is wrong, or what would never happen that way in a real shop.
- **Governance participation:** TIOTF operates through its membership. Working groups, standards committees, and advisory councils all depend on engaged members who care about the industry.

## Glossary of Terms You Will Encounter

| Term           | Plain-language meaning                                                                                                                                                                                            |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Workorder      | The record that tracks a vehicle service job from start to completion — what was done, by whom, with what parts, at what price. Always written as one word.                                                       |
| Capability     | A discrete piece of functional behavior delivered as a unit of work. Roughly: one feature or screen that a user can actually use.                                                                                 |
| Sprint         | A batch of related capabilities delivered together, typically focused on one domain area.                                                                                                                         |
| Persona        | A named user role (e.g., Service Advisor, Technician) that represents a real job and its associated workflows in the system.                                                                                      |
| Journey        | An end-to-end workflow that connects multiple personas and steps — for example, "customer intake to estimate" or "work order execution and completion."                                                           |
| Journey Family | A cluster of related journeys that share actors, domain area, or operational context.                                                                                                                             |
| Estimate       | A quoted price and scope of work presented to a customer before work begins. Requires customer approval before it can be converted to a work order.                                                               |
| SDK            | Software Development Kit — the defined contract between the Durion frontend and backend. It specifies what data can be requested, what responses look like, and how changes are communicated across the boundary. |
| TIOTF          | Tire Industry Open Technology Foundation — the non-profit that governs Durion and its open-source standards.                                                                                                      |



| Term               | Plain-language meaning                                                                                                              |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tenant cell</b> | An isolated runtime environment for one paying customer. Each customer's data and configuration is completely separate from others. |

*This is a first draft. We will elaborate on each section together based on your questions and feedback. Nothing here is final — the same spirit of iterative learning that drives the software also drives how we document it.*